

DL-06-T7

Revised LLM-Agent Architecture

DiscountMate Shopping Assistant

Technical Design Specification

Author:	Sunain Mushtaq
Document:	DL-06-T7
Type:	Technical Design Specification

1. Overview

This document defines the architecture for converting the existing Recipe RAG chatbot into a modular **DiscountMate Shopping Assistant**. It covers the folder structure, LLM-agent responsibilities, LangGraph workflow, tool boundaries, JSON contracts, data flow, and how the existing Recipe RAG is reused.

Core Design Shift

The previous chatbot assumed every request was recipe-related. The new design introduces a higher-level LLM agent that routes requests to the correct specialised tool.

Before	After
User → Recipe Chatbot → Recipe Retrieval → LLM → Recipe Answer	User → DiscountMate Assistant → LLM Agent (intent + entity) → LangGraph → Tool → Backend / DB → LLM Response

LLM responsibilities in the new design: (1) understand user intent, (2) extract entities, (3) choose the right tool, (4) validate arguments, (5) interpret the tool result, (6) generate the final response.

2. High-Level System Architecture

The diagram below shows how a user message travels from the frontend through to the database and back.



Figure 1 — Eight-component pipeline: Frontend → Express → Flask → LLM Agent → LangGraph → Tool Layer → MCP → Backend Services → PostgreSQL

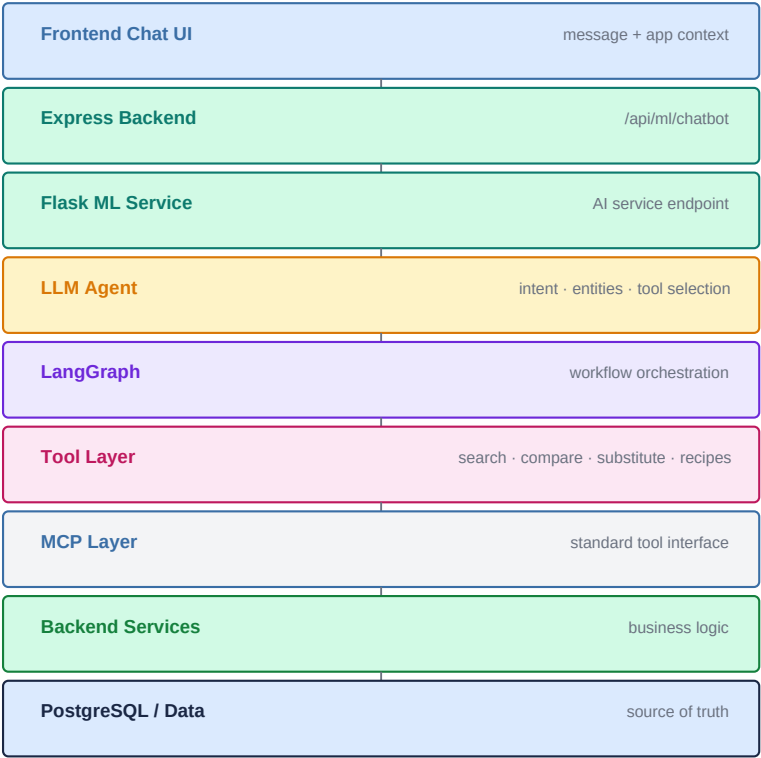


Figure 2 — Layered stack view of the same pipeline

3. Proposed Folder Structure

Each module has a single, clearly defined responsibility.

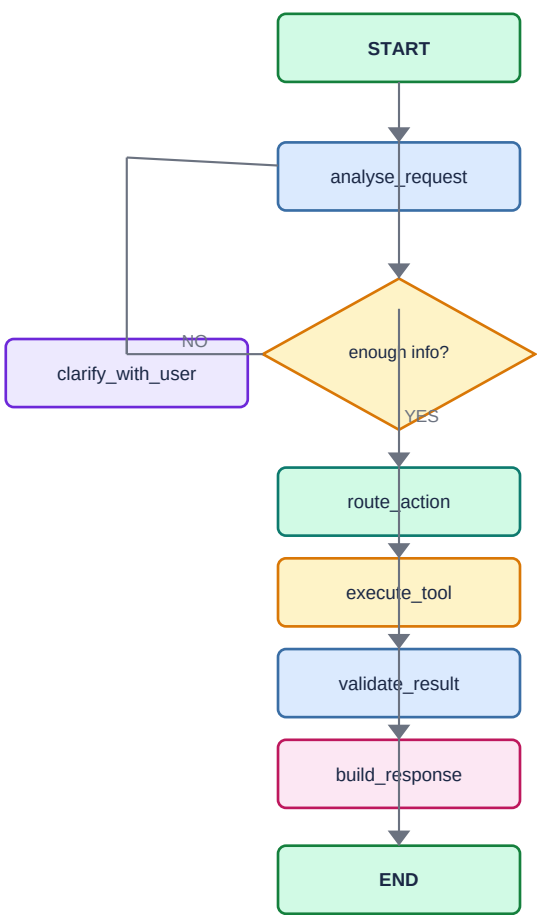
```
Backend/ml-service/
├── app.py
├── requirements.txt
├── chatbot/
│   ├── __init__.py
│   ├── agent.py           ← LLM logic
│   ├── graph.py          ← LangGraph workflow
│   ├── state.py          ← conversation state
│   ├── prompts.py
│   ├── schemas.py        ← Pydantic models
│   ├── config.py
│   └── tools/
│       ├── product_search.py
│       ├── price_comparison.py
│       ├── product_details.py
│       ├── product_substitution.py
│       └── recipe_search.py ← wraps Recipe RAG
│   ├── services/
│       ├── backend_client.py
│       ├── product_matcher.py
│       └── session_service.py
│   ├── evaluation/
│       └── tool_tests.py
├── recipe_rag/
│   ├── rag_pipeline.py ← kept as-is
│   ├── gcs_loader.py
│   └── index/
```

Figure 3 — Module layout inside ml-service/

File / Folder	Responsibility
agent.py	Communicates with the LLM. Never queries the database directly.
graph.py	Contains the LangGraph workflow — orchestration only.
state.py	Tracks conversation state across turns (product_id, tool_result, etc.).
schemas.py	Pydantic models for all requests and responses.
tools/	One file per business capability. No cross-tool dependencies.
services/	Infrastructure layer — backend HTTP client, product matcher.
recipe_rag/	Existing Recipe RAG kept intact. recipe_search.py wraps it.

4. LangGraph Workflow

LangGraph orchestrates the workflow — it does **not** perform product search. It gives deterministic control over what happens after the LLM makes a decision.



analyse_request

Determines the action and extracts structured arguments from the user message.

```
{
  "action": "price_comparison",
  "arguments": {
    "product_name": "Coke Zero",
    "pack_size": "2L"
  }
}
```

route_action

Maps the action name to the correct tool function.

check_arguments

Decides if enough information exists to proceed, or whether clarification is needed.

execute_tool

Runs the selected tool. The LLM never calculates prices or queries the database.

validate_result

Confirms the tool succeeded, records exist, prices are valid, and the schema is correct.

build_response

The LLM converts the structured result into readable natural language.

5. Price Comparison Flow

This is the first fully implemented workflow. It shows how two tools chain together to answer a price query.

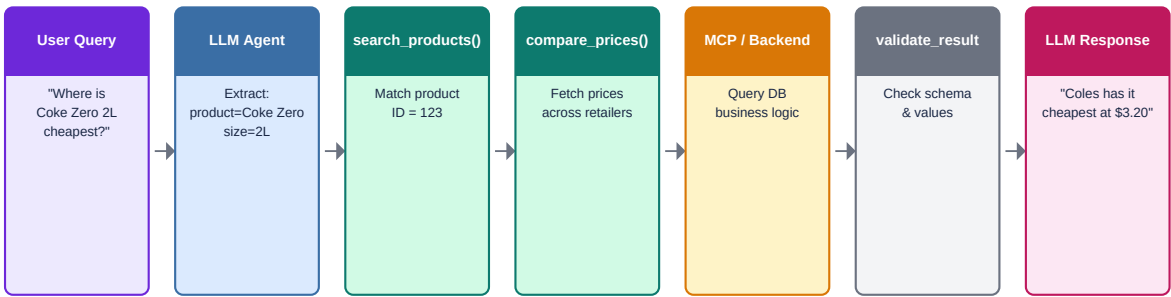
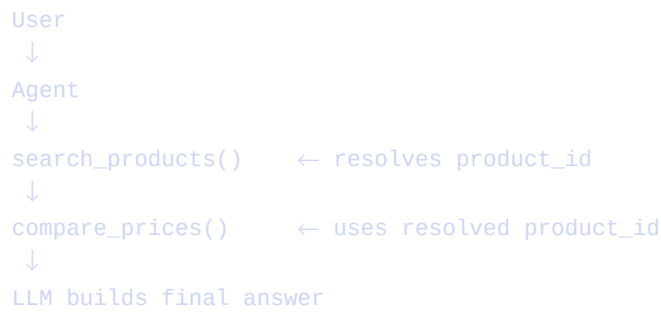


Figure 4 — End-to-end flow for "Where is Coke Zero 2L cheapest?" → "Coles has it cheapest at \$3.20."

Multi-Action Flow

For queries like *Find Coke Zero and tell me where it is cheapest*, two tools run in sequence:



6. Tool Specifications

Each tool owns exactly one business capability. Tools never contain NLP, LLM configuration, or SQL generation.

search_products

Find products matching user-supplied criteria. Must run before compare_prices when no product_id is known.

Input	Output
<pre>{ "product_name": "Coke Zero", "brand": "Coca-Cola", "pack_size": "2L", "retailer": null }</pre>	<pre>{ "products": [{ "product_id": "123", "name": "Coca-Cola Coke Zero 2L", "pack_size": "2L" }] }</pre>

compare_prices

Compare prices for an already-resolved product. Receives a product_id — it does NOT decide what the user means.

Input	Output
<pre>{ "product_id": "123" }</pre>	<pre>{ "product_id": "123", "prices": [{"retailer": "Coles", "price": 3.20}, {"retailer": "Woolworths", "price": 3.50}], "cheapest": {"retailer": "Coles", "price": 3.20} }</pre>

get_product_details

Return name, brand, category, pack size, image, and retailer info for a known product.

Input	Output
<pre>{ "product_id": "123" }</pre>	<pre>{ "name": "...", "brand": "...", "category": "...", ...</pre>

find_substitutes

Suggest cheaper or equivalent alternatives. Stays separate from price comparison.

Input	Output
<pre>{ "product_id": "123", "criteria": {"cheaper": true} }</pre>	<pre>{ "substitutes": [...] }</pre>

search_recipes

Wraps the existing Recipe RAG unchanged. No MCP needed — runs as a local tool.

Input	Output
<pre>{ "query": "pasta carbonara" }</pre>	<pre>{ "recipes": [...] }</pre>

7. Agent ↔ Tool JSON Contract

Every tool uses the same envelope structure so integration is consistent and error handling is predictable.

Agent Request	Tool Response
<pre>{ "action": "price_comparison", "arguments": { "product_name": "Coke Zero", "pack_size": "2L" }, "context": { "product_id": null, "retailer": null } }</pre>	<pre>{ "success": true, "tool": "compare_prices", "data": { "product_id": "123", "product_name": "Coca-Cola Coke Zero 2L", "prices": [{"retailer": "Coles", "price": 3.20}] }, "error": null }</pre>

8. MCP Layer

MCP sits between the chatbot tools and the backend. It exposes clean **business actions**, not raw database operations.

■ Avoid — DB-level	✓ Preferred — Action-level
<pre>get_products_table() run_sql() get_pricing_rows()</pre>	<pre>search_products() compare_prices() get_product_details() find_substitutes()</pre>

The existing Recipe RAG does **not** need MCP internally at first — `search_recipes()` calls it directly as a local tool. Product and pricing tools route through MCP.





9. Application Context Support

If the user is already viewing a product page, the frontend can pass that context so the LLM does not need to guess what *this* refers to.

```
// Frontend sends:
{
  "message": "Where is this cheapest?",
  "context": {
    "screen": "product_details",
    "product_id": "123"
  }
}
```

Design for this now — even if the frontend doesn't use it yet. The context field in the JSON contract makes it easy to add later without changing the agent schema.

10. Responsibility Matrix

Component	Responsible for
Frontend	User input, app context, displaying results
Express	API gateway — routes requests to Flask
Flask	AI service endpoints
LLM	Intent extraction, entity recognition, tool selection, response wording
LangGraph	Workflow control — what happens after each LLM decision
Tool	One specific business capability per file
MCP / API	Stable, action-level interface to backend data
Backend	Business and data logic
Database	Source of truth for products, prices, stock
Recipe RAG	Recipe retrieval only — unchanged

11. Key Architectural Rule

Design Rule: LLMs decide what information is required. Tools retrieve that information. DiscountMate data remains the source of truth.

This prevents:

- LLM inventing prices or products
- LLM generating SQL queries
- LLM depending directly on database schemas
- Business logic leaking into the AI layer

12. Implementation Plan

Phase 1 — Initial Build

- **agent.py** + **graph.py** + **state.py** — LLM agent and LangGraph scaffold
- **schemas.py** — Pydantic models for all tool contracts
- **search_products** + **compare_prices** — first two tools
- **recipe_search** wrapping the existing Recipe RAG

Phase 2 — Extend

- **get_product_details** — enriched product information
- **find_substitutes** — cheaper/equivalent alternatives
- Application context from frontend (`product_id` passthrough)
- Evaluation suite — `tool_tests.py`, `test_queries.json`

The main architectural advantage: new tools can be added without redesigning the agent or LangGraph workflow. Each tool is an independent module.